Protocolo HTTP

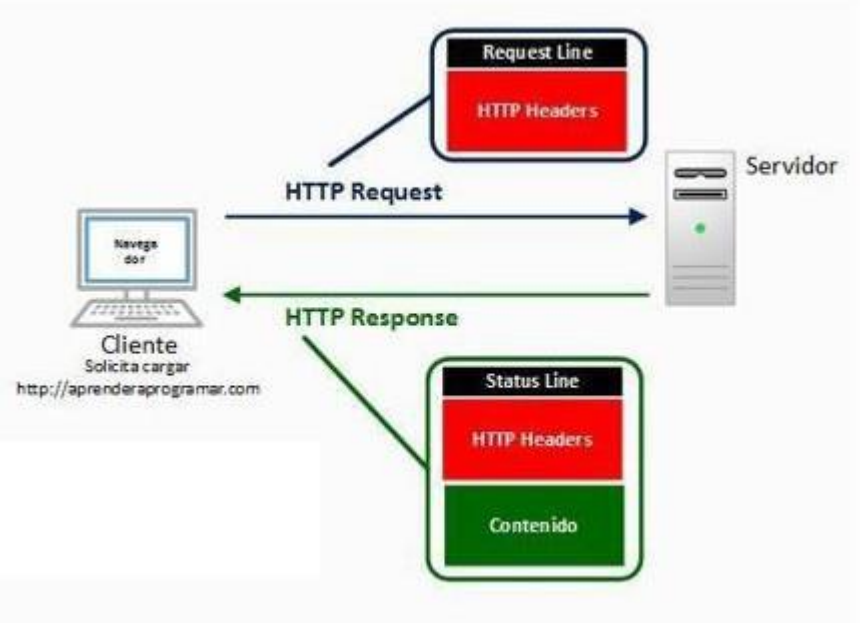
Es un protocolo de comunicaciones estándar que comunica servidores, proxies y clientes. Permite la transferencia de documentos web, sin importar cual es el cliente o cual es el servidor. Es un protocolo basado en el esquema petición/respuesta.

Funcionamiento:

Un usuario escribe una URL en su navegador, indicando el protocolo, servidor y recurso que quiere obtener, el servidor procesa dicha información y devuelve un mensaje de respuesta, normalmente con el HTML de la página que se va a visualizar.

Ejemplo: http://www.sena.edu.co

Esquema Protocolo HTTP

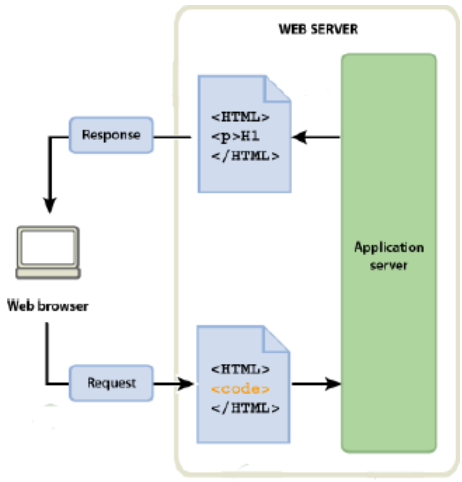


Métodos envío de datos

- **GET**: Petición donde se solicitan datos al servidor.
- **POST**: Petición donde se envían datos al servidor para su procesamiento.
- **PUT**: Petición donde se envían datos al servidor, normalmente para actualizar datos.
- **DELETE**: Petición donde se envían datos para eliminar.

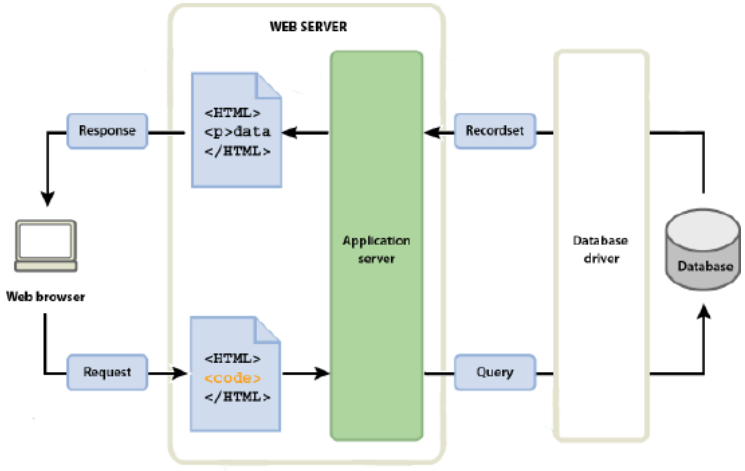
Procesamiento de web dinámicas

Proceso mediante el cual se genera un documento html a partir de una solicitud de un usuario. El documento html generado dependerá del tipo de solicitud, parámetros dados en la petición, etc.



Peticiones que requiere acceso a una base de datos

Las aplicaciones web permiten poder acceder a recursos que se encuentran en bases de datos. El uso de una base de datos para almacenar contenido permite separar el diseño de la aplicación web, del contenido que se desea mostrar a los usuarios del sitio. En lugar de escribir archivos HTML individuales para cada página, sólo se necesita escribir una página —o plantilla— para presentar los datos de la base de datos al usuario. También de forma dinámica se puede presentar formularios HTML que añadan o modifiquen información en la base de datos. En la siguiente imagen se puede observar la representación de peticiones que requieren acceder a bases de datos.



Desarrollo aplicaciones Web en Python

Inicialmente vamos a trabajar con un micro-framework de Python llamado ***Flask*** para el desarrollo de aplicaciones web. ***Flask*** está basado en ***Werkzeug***, ***Jinja 2*** y buenas intenciones. Mediante Flask podemos construir aplicaciones web y servicios Restful con Python de una forma extraordinariamente sencilla. Con pocas líneas podemos llegar a tener un servicio Restful funcionando.

Características de Flask

Ser un ***micro-framework*** no es indicativo de que es para aplicaciones pequeñas, por el contrario, significa que añade pocos elementos, permitiendo estar más cercano al lenguaje base, mejorando el rendimiento de la aplicación.

Estructura de un proyecto Flask

Por convención los proyectos ***Flask*** tienen la siguiente estructura:

- **static**, para los ficheros estáticos, como hojas de estilos, imágenes, javascript, etc.
- **templates**, para los documentos html.
- **app.py:** Archivo Python que inicia la aplicación (Puede llamarse diferente de app.py)

Requisitos para utilizar el framework Flask

Flask depende de librerías externas como ***Werkzeug*** y ***Jinja2***. Werkzeug es un toolkit para aplicaciones ***WSGI***, que es una interface entre aplicaciones Python y servidores web. ***Jinja2*** es un engine para el renderizado de plantillas (o templates) web. Para poder utilizar Flask debes de tener, al menos, Python 2.6 instalado. Flask también funciona con Python 3.

Creación primer Proyecto web con Flask

- Crear una carpeta de trabajo.
- Desde visual studio code abrir la carpeta.

En la raíz de la carpeta crear un archivo llamado app.py

En la consola de Visual Studio Code: pip install Flask

```
SENA@DESKTOP-HOG20R5 MINGW64 /d/Documentos/SENA2024/FICHAS/ADSO 2669742/Ac
tividades/Febrero 21 de 2024/InicioWebFlask
$ pip install Flask
```

A continuación, un breve código que incluye los elementos necesarios para ejecutar una aplicación web.

app.py > ...

```python
from flask import Flask

#Crear el objeto de Tipo Flask
app = Flask(__name__)

#Raiz del Sitio
@app.route("/")
def inicio():
    return "Hola Mundo..."

#Iniciar la Aplicación
if __name__=="__main__":
    app.run(port=3000, debug=True)

```

En el código anterior se crea una ***instancia de Flask*** y se inicia la aplicación por el **puerto 3000** y en modo ***debug*** para que se recargue automáticamente si hay actualizaciones en el código y muestre errores.

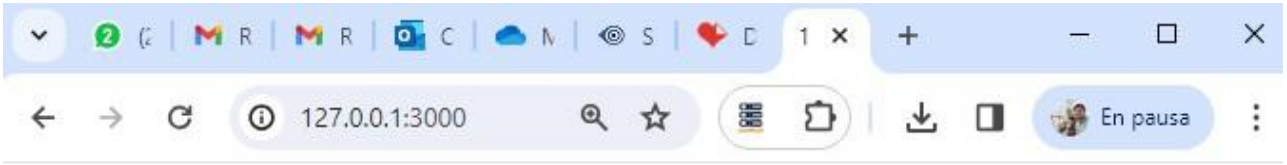
El siguiente código contiene una función llamada inicio que contiene un decorador **@app.route('/')** para mapear la ruta de la URL de la función. Adicionalmente la ruta apunta a la raíz del proyecto.

```python
#Raiz del Sitio
@app.route("/")
def inicio():
    return "Hola Mundo..."
```

Si ejecutamos la aplicación podemos ver en la terminal el proceso realizado como se muestra en la siguiente imagen:

```
PS D:\Documentos\SENA2024\FICHAS\ADSO 2669742\Actividades\Febrero 21 de 2024\InicioWebFlask> & C:/U
sers/SENA/AppData/Local/Programs/Python/Python312/python.exe "d:/Documentos/SENA2024/FICHAS/ADSO 26
69742/Actividades/Febrero 21 de 2024/InicioWebFlask/app.py"
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production W
SGI server instead.
 * Running on http://127.0.0.1:3000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 670-766-342
```

En el código anterior informa la url para poder ingresar al sitio que en este caso es la http://1127.0.0.1:3000. Por lo anterior vamos a mostrar el resultado en el navegador:

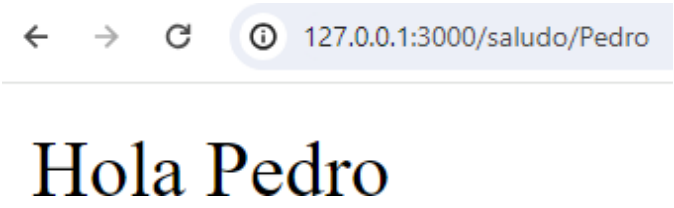


Hola Mundo...

Ahora podemos agregar una función a otra ruta donde le pasamos un parámetro y ese valor es utilizado en la función:

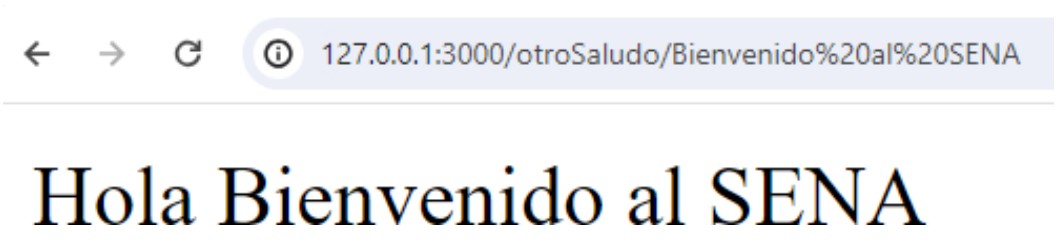
```
11  @app.route("/saludo/<nombre>")
12  def saludo(nombre):
13      return f"Hola {nombre}"
```

Se puede probar en el navegador así:



Se puede crear un decorador donde se pasa un parámetro y se le puede indicar de que tipo como se muestra en el ejemplo:

```
15  @app.route("/otroSaludo/<string:mensaje>")
16  def otroSaludo(mensaje):
17      return f"Hola {mensaje}"
```



También se pueden recibir argumentos que se pasan por la url y se reciben de la siguiente forma:

```
19  @app.route("/producto/<nombreProducto>")
20  def datosProducto(nombreProducto):
21      codigo = request.args.get('codigo')
22      return f"El {nombreProducto} tiene código {codigo}"
```

En el código anterior estamos usando un objeto de tipo **request** el cual debe ser importado desde la librería **Flask**.

```
app.py > inicio
1   from flask import Flask, request
```

La forma como se pasa en la url es de la siguiente forma:



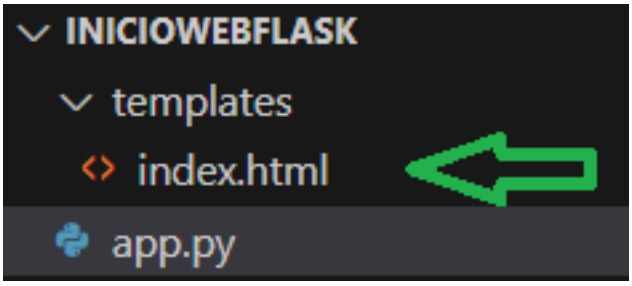
El televisor tiene código 10

Ahora vamos a revisar cómo hacer para que se cargue un documento html. En Python se debe crear una carpeta llamada ***templates*** donde se deben crear los documentos html.

Como ejemplo creamos un documento html llamado index.html en la carpeta **templates**.

```
templates > index.html > ...
1   <!DOCTYPE html>
2   <html lang="en">
3   <head>
4       <meta charset="UTF-8">
5       <meta name="viewport" content="width=device-width, initial-scale=1.0">
6       <title>Mi primer Documento HTML en PYTHON</title>
7   </head>
8   <body>
9       <h1>DESARROLLO DE SITIOS WEB EN PYTHON CON FLASK</h1>
10  </body>
11  </html>
```

En la estructura de carpetas y archivos podemos ver la ubicación del archivo index.html



Ahora vamos a modificar el método que tiene el decorador de la raíz para que al cargar el sitio se muestre la página llamada **index.html**

```
6   #Raiz del Sitio
7   @app.route("/")
8   def inicio():
9       #return "Hola Mundo..."
10      return render_template("index.html")
```

En el código anterior se está utilizando una función llamada **render_template** la cual debe ser importada desde **Flask**.

```
app.py > saludo
1   from flask import Flask, request, render_template
2
```

Al recargar la aplicación ya debe mostrar el resultado del documento index.html.



Como ejemplo crear un Documento HTML que muestra una tabla y se crea una función en la aplicación para cargarla.

Función en la aplicación:

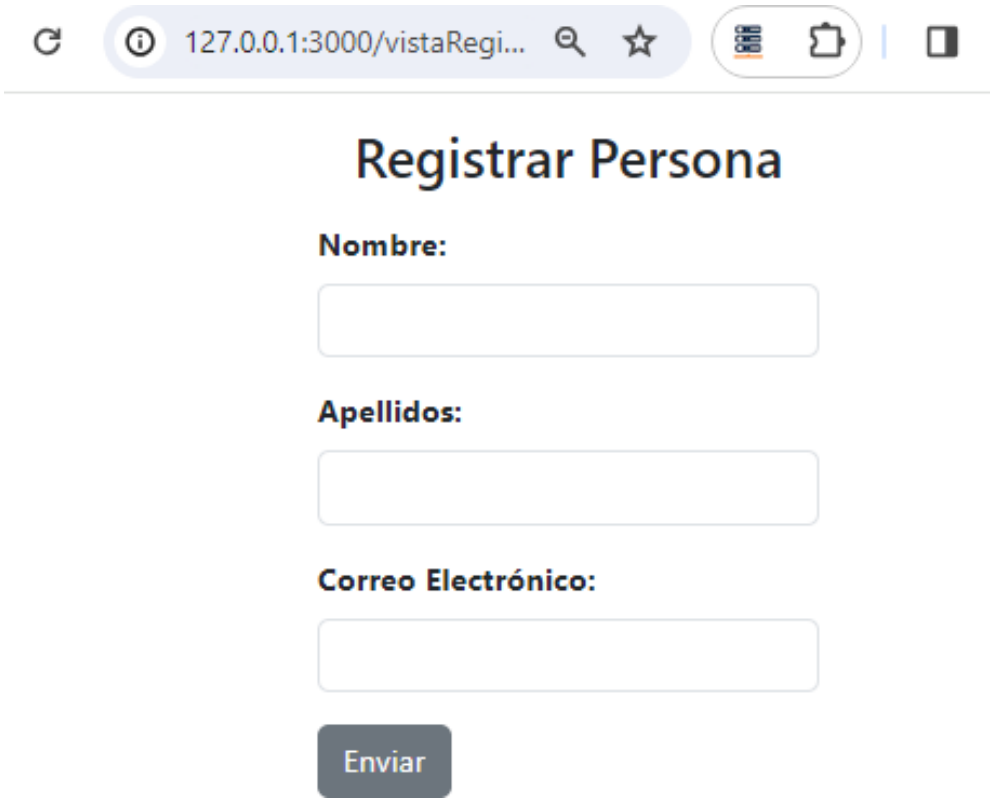
```
25  @app.route("/tabla")
26  def mostrarTabla():
27      return render_template("tabla.html")
```

Resultado:



Ejemplo documento html con un formulario y como recibir los datos.

Vista del Formulario



Código html del formulario:

```
<body>
    <div class="container">
        <div style="margin: 0 auto" class="w-50">
            <h2 class="text-center mt-3">Registrar Persona</h2>
            <form action="/registrarPersona", method="post" style="margin: 0 auto">
                <div class="mb-3 mt-3">
                    <label for="txtNombres" class="form-label fw-bold">Nombre:</label>
                    <input type="text" class="form-control" id="txtNombres"
                    name="txtNombres" required>
                </div>
                <div class="mb-3 mt-3">
                    <label for="txtApellidos" class="form-label fw-bold">Apellidos:</label>
                    <input type="text" class="form-control" id="txtApellidos"
                    name="txtApellidos" required>
                </div>
                <div class="mb-3 mt-3">
                    <label for="txtCorreo" class="form-label fw-bold">Correo Electrónico:</label>
                    <input type="email" class="form-control" id="txtCorreo"
                    name="txtCorreo" required>
                </div>
                <button type="submit" class="btn btn-secondary">Enviar</button>
            </form>
            <p class="text-center">{{mensaje}}</p>
        </div>
    </div>
</body>
```

Se crea una ruta en Python para mostrar la vista inicialmente y otra ruta para que reciba los datos del formulario y haga las acciones que tenga que hacer.

Función con la ruta para mostrar el formulario:

```
43  @app.route("/vistaRegistrarPersona")
44  def vistaRegistrarPersona():
45      return render_template("frmPersona.html")
46
```

Ahora la función la ruta para recibir los datos del formulario

```
31  @app.route("/registrarPersona", methods=['POST'])
32  def registrarPersona():
33      if request.method=="POST":
34          nombre = request.form["txtNombres"]
35          apellido=request.form["txtApellidos"]
36          correo=request.form["txtCorreo"]
37          persona=[nombre,apellido,correo]
38          listaPersonas.append(persona)
39          return render_template("frmPersona.html", mensaje="Persona Agregada correctamente")
40      else:
41          return render_template("frmPersona.html", mensaje="No permitido")
```

En la función de **registrarPersona** se crea una lista con los datos y se agregan a otra lista a manera de ejemplo. Adicionalmente retorna a la vista con un mensaje. Ese parámetro llamado mensaje se recibe en la vista y se muestra el valor.

Bibliografía:

- Tutorial de Flask. https://j2logo.com/tutorial-flask-espanol/
- Nolasco Valenzuela, J. S. (2018). Python: aplicaciones prácticas. RA-MA Editorial. https://elibro-net.bdigital.sena.edu.co/es/lc/senavirtual/titulos/106523
- Lopez Pascual, Walter L. (2020). Comenzando con Docker. https://books.google.com.co/books?id=aPz3DwAAQBAJ&printsec=frontcover&hl=es#v=onepage&q&f=false
- Sitio oficial Flask: https://flask.palletsprojects.com/en/2.0.x/

CONTROL DEL DOCUMENTO

	Nombre	Cargo	Dependencia	Fecha
Autor (es)	César Marino Cuéllar Chacón	Instructor	CTPI-CAUCA	28-02-2025